

AGENTIC MULTIMODAL RAG SYSTEM ARCHITECTURE REPORT

Take-Home Submission

1) Executive Summary

This application is a full-stack, user-scoped, agentic Retrieval-Augmented Generation (RAG) system designed for document-grounded question answering with optional web fallback.

It combines a FastAPI backend, Streamlit frontend, LangGraph orchestration, ChromaDB vector storage, sentence-transformer embeddings, a local cross-encoder reranker, and Groq-hosted LLM/vision models.

Core design goals in the implemented architecture are:

- Strong grounding on user-uploaded private documents.
- Robust handling for multimodal and structured data (PDF images/tables, DOCX images, XLSX/XLS/CSV tabular content).
- Query optimization before retrieval (multi-query expansion + reranking).
- Reliable session continuity via short-term and long-term memory.
- Safety and confidence controls (grounding ratio guardrail).

2) High-Level System Topology

Frontend:

- Streamlit app at port 8501.
- Handles auth UX, document upload UI, chat UI, source rendering, and voice capture.

Backend:

- FastAPI app at port 8000.
- Routers:
 - /auth (username/password + Google OAuth)
 - /documents (upload/list/delete with full cleanup)
 - /chat (agentic RAG execution and session persistence)
 - /voice (audio transcription)

Persistent Stores:

- SQLite:
 - users (authentication)
 - conversation_history (long-term memory)
 - chat_session_meta (session titles)
- ChromaDB:
 - rag_documents_raw (raw chunk nodes)
 - rag_documents_summary (summary nodes: doc/section/table/image)

Model Layers:

- Embeddings: all-MiniLM-L6-v2 (Hugging Face sentence-transformers)
- Reranker: cross-encoder/ms-marco-MiniLM-L-6-v2 (local CPU)
- LLM: Groq text model (configured by GROQ_MODEL)
- Vision summarization: Groq vision model (configured by VISION_MODEL)
- STT: faster-whisper (local CPU int8)

3) End-to-End Flow (User Question)

Step A: User sends question from Streamlit

- UI calls POST /chat with:
 - message

- session_id
- include_long_term_memory flag

Step B: Chat API composes memory context

- Short-term memory: recent turns from active session_id.
- Optional long-term memory: recent turns from prior sessions only (active session excluded to avoid duplication).

Step C: Agent execution

- run_agent(query, user_id, conversation_history)
- LangGraph flow:
 1. retrieve node
 2. router node decides path
 - rag_answer (document-grounded answer)
 - doc_summary (if question references specific document name)
 - web_search (fallback)

Step D: Response assembly

- API returns:
 - answer text
 - source references (source, score/excerpt/url/page)
 - used_web_search flag
 - latency_ms
 - token_usage (prompt/completion/total when available)

Step E: Persistence

- user and assistant turns saved in SQLite conversation_history.

4) Ingestion Architecture (Document Processing)

4.1 Upload and User Isolation

- Upload endpoint stores each file in user-specific folder:
data/documents/user_<user_id>/filename
- Ingestion is user-scoped: all stored vector nodes carry user_id metadata.

4.2 Supported Formats

- PDF, DOCX, DOC, TXT, MD, XLSX, XLS, CSV.

4.3 Loader Strategy by Format

PDF:

- Primary path: PyMuPDF text extraction (fast and reliable for digital PDFs).
- Multimodal path: unstructured.partition.pdf (hi_res) with:
 - OCR support for scanned content,
 - table extraction,
 - image payload extraction.
- The system can route to multimodal mode when:
 - PDF_INGESTION_MODE requests rich path, or
 - image summaries are enabled and images are detected, or
 - OCR fallback is needed.

DOCX:

- Text via Docx2txtLoader.
- Embedded images extracted from DOCX zip package (word/media/*), then summarized by vision model.

TXT/MD:

- TextLoader.

XLSX/XLS/CSV:

- Parsed row-by-row into retrieval-friendly records.
- Header detection + normalization + dedupe.

- Date normalization with multi-format variants (e.g., 2020-07-14, 7/14/2020, 7/14/20, month-name forms).
- Derived facts are generated for semantically important row patterns, e.g.: "Patient X was admitted for Arthritis on 2020-07-14."
- Row-level metadata includes row_number and content_type=table_row.

5) Chunking Strategy (How Chunking Is Done)

Chunking happens after loader output (raw_docs) is assembled.

Implementation details:

- Splitter: RecursiveCharacterTextSplitter.
- Effective parameters are enforced as:
 - chunk_size = max(config.CHUNK_SIZE, 800)
 - chunk_overlap = max(config.CHUNK_OVERLAP, 150)
- This means even if env CHUNK_SIZE/OVERLAP are lower, the system guarantees an 800/150 floor for better semantic coherence.
- Separators used in order: paragraph, newline, sentence-ish, space, character.

Each chunk is stamped with metadata:

- source, file_path, user_id
- document_id (stable UUIDv5 by user_id+filename)
- node_type=raw_chunk
- parent_id=document_id
- chunk_index, total_chunks

Why this chunking is useful:

- Improves retrieval precision by balancing chunk granularity and context width.
- Overlap reduces boundary information loss.
- Stable metadata enables robust lifecycle operations (full-document retrieval, precise deletion, summary-parent expansion).

6) Summary Architecture (How Summaries Are Built)

The system maintains summary nodes separate from raw chunks.

6.1 Summary Types

- document_summary: whole-document summary.
- section_summary: per-section summary when section titles are detected and ENABLE_SECTION_SUMMARIES=true.
- table_summary: generated from extracted table text (primarily PDF multimodal paths).
- image_summary: generated from extracted image content (PDF and DOCX image assets).

6.2 Generation

- Text summaries use Groq text model with concise retrieval-oriented prompts.
- Image summaries use Groq vision model on extracted images.

6.3 Storage

- Raw chunks stored in Chroma collection: rag_documents_raw.
- Summary nodes stored in Chroma collection: rag_documents_summary.

Summary node metadata includes:

- source, file_path, user_id, document_id, parent_id
- node_type and type-specific keys (section_id/title, table_index, image_index/path/mime, page range, etc.)

Why summaries are useful:

- Faster high-level document matching (summary-first retrieval).
- Better recall for abstract questions ("summarize", "main findings").
- Better multimodal grounding from images/tables that raw text alone may miss.

7) Retrieval and Query Optimization (Detailed)

The retrieval pipeline is intentionally multi-stage.

7.1 Query Expansion (Optimization Step 1)

- The user query is expanded into multiple semantically distinct variants using LLM.
- If expansion fails, system safely falls back to original query only.

Benefit:

- Improves recall when documents use alternative phrasing.

7.2 Summary-First Retrieval (Optimization Step 2)

- For each query variant, similarity search runs against summary collection.
- Top matching summaries identify likely document_ids.

Benefit:

- Narrows search space to the most relevant documents early.

7.3 Parent-Child Expansion (Optimization Step 3)

- For selected document_ids, all raw chunks of those documents are pulled.
- Additional targeted raw similarity searches run per variant.
- Results are deduplicated by content key.

Benefit:

- Keeps broad recall while focusing ranking on meaningful candidate pool.

7.4 Structured Date-Intent Filter (Optimization Step 4)

- For tabular queries that contain intent/date patterns (e.g., admitted on 7/14/20), candidates are filtered to rows where intent and date co-occur semantically.
- If no exact row matches condition+date intent, a structured no-match notice is injected to prevent hallucinated entity answers.

Benefit:

- Fixes common table-QA failure mode (admission date vs discharge date confusion).

7.5 Cross-Encoder Reranking (Optimization Step 5)

- Candidate (query, passage) pairs are reranked locally with cross-encoder model.
- Top-k reranked passages become final context.

Benefit:

- Significant precision improvement over embedding score alone.

7.6 Final Context Shaping

- Top image summaries associated with top-ranked documents may be appended as supplemental visual context.

8) Agent Routing and Answer Generation

Node: retrieve

- Produces retrieved_chunks and optionally matched document if explicit file-name reference exists.

Node: router

- If specific document matched -> doc_summary path.
- Else if top chunk score $\geq$ threshold (-3.0) -> rag_answer path.
- Else -> web_search path.

Node: rag_answer

- Uses RAG system prompt + conversation history + formatted context.
- Returns grounded answer + source excerpts/pages.

Node: doc_summary

- Uses document summaries + section summaries + visual summaries + raw excerpts.
- Better for broad "summarize this doc" or named-document questions.

Node: web_search

- Uses DuckDuckGo snippets + Groq model synthesis.

9) Grounding, Safety, and Confidence Controls

- Prompt-level safety instructions:
 - no fabrication,
 - explicit insufficiency statements,
 - unsafe request refusal.
- Grounding support ratio:
 - lexical overlap signal between answer tokens and retrieved context.
 - if below GROUNDING_MIN_RATIO, appends low-confidence grounding disclaimer.
- Structured no-match notice from retrieval:
 - explicitly informs answer generation when exact table row not found.

10) Memory System

Short-term memory:

- Recent messages from active session_id are always loaded.
- Preserves conversational continuity in current chat.

Long-term memory:

- SQLite stores all turns.
- Optional cross-session context from recent previous sessions can be injected.

Session APIs:

- list sessions, fetch session messages, rename session, delete session.

11) Data Lifecycle and Deletion Semantics

When document deletion is requested:

- All raw and summary vector nodes for that source are removed.
- Local file is deleted from user folder.
- Multimodal asset directory (.multimodal_assets/<document_id>) is removed.
- Conversation rows referencing this source are removed.
- Vectorstore caches and agent graph are reset.

Result:

- Document context is removed from retrieval and historical references.

12) Storage Model Summary

SQLite:

- users
- conversation_history
- chat_session_meta

ChromaDB (persist_directory = CHROMA_DB_PATH):

- rag_documents_raw
- rag_documents_summary

Filesystem:

- data/documents/user_<id>/... uploaded files
- data/documents/user_<id>/... .multimodal_assets/<document_id>/... extracted images

- vectorstore/... persistent embeddings/index

13) Runtime and Deployment

Local:

- Backend: `uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload`
- Frontend: `streamlit run frontend/streamlit_app.py --server.port 8501`
- Optional helper: `./start.sh`

Docker:

- `docker compose up --build`
- API container exposes :8000 with healthcheck.
- Frontend container exposes :8501 and talks to API internally via `API_BASE=http://api:8000`.
- Browser OAuth URL uses `BROWSER_API_BASE=http://localhost:8000`.

14) Why This Architecture Works Well

- Separation of raw vs summary collections provides both precision and recall.
- Parent-child linking via `document_id` enables scalable and explainable retrieval.
- Multimodal extraction adds answerable context beyond plain OCR/text.
- Structured tabular retrieval pipeline improves factual QA for spreadsheet-style datasets.
- User-scoped metadata ensures multi-user isolation in one deployment.
- Cleanup semantics prevent stale context after deletes.

15) Known Tradeoffs and Future Improvements

Current tradeoffs:

- Cross-encoder adds CPU latency per query.
- First model downloads can be slow (HF embeddings/reranker, whisper).
- Groq token/rate limits affect summarization and chat under free-tier constraints.

Potential improvements:

- Async/background ingestion jobs with progress polling.
- Hybrid lexical + vector retrieval for even stronger table/entity matching.
- Add structured parser profiles for domain-specific spreadsheets.
- Add optional answer verifier stage for high-stakes QA.
- Frontend image/table citation rendering improvements.

End of report.